CANTINA

Infrared Operations & Future Vaults

Security Review

Cantina Managed review by:
Alireza Arjmand, Lead Security Researcher
Cryptara, Security Researcher

November 12, 2025

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3

3 Findings **4**
- 3.1 High Risk . . . 4
  - 3.1.1 Unaccrued rewards lost when updating reward duration . . . 4
- 3.2 Low Risk . . . 4
  - 3.2.1 Inability to forcefully remove malicious reward tokens . . . 4
- 3.3 Informational . . . 4
  - 3.3.1 Redundant reward calculations and state mutations within loop . . . 4
  - 3.3.2 Unnecessary period reset in `_setRewardsDuration` . . . 5
  - 3.3.3 Redundant reward calculation in `addIncentives` . . . 6
  - 3.3.4 Missing Cleanup of `rewardDecimals` Mapping . . . 6
  - 3.3.5 Unchecked Decimals Value . . . 6
  - 3.3.6 Unused Function and Commented Logic . . . 7

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Infrared simplifies interacting with Proof of Liquidity with liquid staking products such as iBGT and iBERA.

From Oct 14th to Oct 15th the Cantina team conducted a review of infrared-contracts on commit hash a18fb7d. The team identified a total of **8** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	0	0	0
Low Risk	1	1	0
Gas Optimizations	0	0	0
Informational	6	3	3
Total	**8**	**5**	**3**

2.1 Scope

The security review had the following components in scope for infrared-contracts on commit hash a18fb7d:

```
├── PR 641 (https://github.com/infrared-dao/infrared-contracts/pull/641)
│   └── script
│       └── InfraredMultisigGovernance.s.sol
└── PR 642 (https://github.com/infrared-dao/infrared-contracts/pull/642)
    └── src
        └── core
            └── MultiRewards.sol
```

3 Findings

3.1 High Risk

3.1.1 Unaccrued rewards lost when updating reward duration

Severity: High Risk

Context: InfraredMultisigGovernance.s.sol#L331-L377, MultiRewards.sol#L405-L434

Description: In `MultiRewards.sol`, the `_setRewardsDuration` function can be called by the governor to define a new reward duration. Once updated, the contract begins a new reward period using the new duration and updates both `lastUpdateTime` and `periodFinish` for the given reward token.

However, because `lastUpdateTime` is updated without first calling `updateReward(address(0))`, any unaccrued yield up to that point will be lost. This prevents pending rewards from being attributed to users, while leaving `rewardPerTokenStored` unaffected.

Recommendation: *Note: The audited commit on Cantina already has this issue resolved, the issue was disclosed and solved by the client before the start of the audit*.

Either:

1. Call `updateReward(address(0))` at the beginning of `_setRewardsDuration` to ensure all rewards are accrued before modifying the duration, or...
2. Make sure the governor accrues the rewards each time before updating the reward duration.

Solution (1) requires an on-chain change and applies only to newly deployed vaults, whereas solution (2) can be implemented off-chain for vaults already deployed.

Infrared:

- Vault fix in commit 5c7b970c.
- Follow-up operational fix in commit 3335cec8.

Cantina Managed: Verified the fixes.

3.2 Low Risk

3.2.1 Inability to forcefully remove malicious reward tokens

Severity: Low Risk

Context: MultiRewards.sol#L312

Description: In `Infrared.sol`, the `_removeReward` function is intended to be called when a reward token is deemed malicious, as outlined in the contract comments. However, the function currently enforces the following restriction:

```
require(block.timestamp >= rewardData[_rewardsToken].periodFinish);
```

This requirement prevents governance from removing a malicious or failing reward token until the reward period has ended. During that time, users and the protocol remain exposed to the risks associated with the compromised token.

Recommendation: Allow governance to forcefully remove a malicious or failing reward token by removing or bypassing the time restriction check. This ensures that security incidents or token failures can be addressed immediately without waiting for the reward period to finish.

Infrared: Fixed in commit 784d3470.

Cantina Managed: Verified the fix.

3.3 Informational

3.3.1 Redundant reward calculations and state mutations within loop

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Within `updateReward`, the loop repeatedly calls reward-related functions in a way that leads to duplicated calculations:

- `lastTimeRewardApplicable` and `rewardPerToken` are each called directly inside `updateReward`.
- `rewardPerToken` calls `lastTimeRewardApplicable`.
- `earned` is also called, which itself calls `rewardPerToken`, which in turn calls `lastTimeRewardApplicable`.

As a result, for each iteration:

- `lastTimeRewardApplicable` is executed three times,
- `rewardPerToken` is executed twice,
- And `earned` is called once.

This introduces unnecessary duplication and increases the likelihood of inconsistencies with state mutations occurring between these calls. Since reward logic is core functionality, this design also increases maintenance complexity as more use cases are added.

Recommendation: Refactor `updateReward` to reduce redundant calls:

1. Fetch `lastTimeRewardApplicable` once at the start of the iteration.
2. Pass it into a single `rewardPerToken` calculation.
3. Use the result when computing `earned` instead of letting `earned` call `rewardPerToken` and `lastTimeRewardApplicable` again.
4. Perform state updates only after all calculations are completed.

This ensures consistent results, improves maintainability, and avoids unnecessary repeated function calls.

Infrared: Acknowledged. We agree with this finding of inefficiency but will not change for several reasons:

- First, it would require changing functions like earned and rewardPerToken which are already integrated into other contracts of our own and externally.
- Second, last time we changed `updateReward` for efficiency improvements, we accidentally introduced a bug and learned small efficiency improvements are not worth risking on the vault.
- Third, we don't see any scenario where state could mutate between calls.

Cantina Managed: Acknowledged.

3.3.2 Unnecessary period reset in `_setRewardsDuration`

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `_setRewardsDuration` function sets a new `rewardsDuration` while also starting a new reward period. However, starting a new period is not inherently required when updating `rewardsDuration`. This adds unnecessary complexity to the function and creates implicit behavior that may not always align with the intended lifecycle of reward schedules.

Recommendation: Consider simplifying `_setRewardsDuration` so that it only updates the `rewardsDuration` without automatically starting a new reward period. Any redesign should be carefully reviewed, as `rewardsDuration` is used throughout the system and modifications may have broad implications.

Infrared: Acknowledged. Yes, this would be the most simple solution but it would break at least one external view function (`getRewardForDuration`) and possibly open attack vectors with the out of sync state (though this is somewhat speculative).

Cantina Managed: Acknowledged.

3.3.3 Redundant reward calculation in `addIncentives`

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: In VaultManagerLib.sol, the `addIncentives` function re-implements the reward rate adjustment logic found in `_notifyRewardAmount` This ensures that `rewardRate` does not decrease when additional incentives are added for the next duration.

However, the first part of the math (lines 280-284) is redundant. Instead of performing multiple intermediate calculations, the same result can be obtained directly with:

```
totalAmount = reward + leftover + rewardResidual;
```

Recommendation: Simplify the `addIncentives` logic by removing redundant calculations and computing `totalAmount` as above. This reduces code duplication and improves clarity, while still ensuring consistent reward rate handling.

Infrared: Acknowledged. `VaultManagerLib` can only be changed with an upgrade. We will add this to a list of general improvements for our next upgrade (see issue 645).

Cantina Managed: Acknowledged.

3.3.4 Missing Cleanup of `rewardDecimals` Mapping

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `MultiRewards` contract maintains a `rewardDecimals` mapping that caches the decimal precision for each reward token to optimize gas costs during reward calculations. However, the `_removeReward` function fails to clean up this mapping when a reward token is removed from the contract. While the function properly removes the reward token from the `rewardTokens` array and deletes the corresponding `rewardData` entry, it leaves the cached decimal value in the `rewardDecimals` mapping, resulting in stale data that persists indefinitely in the contract's storage.

Recommendation: Add a cleanup statement to remove the decimal mapping entry in the `_removeReward` function. Include `delete rewardDecimals[_rewardsToken];` after the existing cleanup operations to ensure complete removal of all token-related data. This maintains consistency with the contract's data management approach and prevents accumulation of obsolete storage entries.

Infrared: Fixed in commit c8b44ba7.

Cantina Managed: Verified the fix.

3.3.5 Unchecked Decimals Value

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `_addReward` function in the `MultiRewards` contract caches the decimals value of a reward token to optimize reward calculations. However, it does not validate that the decimals value is within a safe range. The contract uses the decimals value to compute a precision factor as $10^{36-\text{decimals}}$. If the decimals value is 36, the precision factor becomes 1, which may be acceptable but results in no scaling. If the decimals value exceeds 36, the calculation underflows, causing the precision factor to revert to zero and breaking the reward logic.

Recommendation: Introduce a check in the `_addReward` function to ensure that the decimals value of the reward token does not exceed 36. If the value is greater than 36, revert the transaction with an appropriate error message. Optionally, consider restricting the decimals value to a reasonable range (such as 6 to 36) to avoid edge cases and ensure consistent reward calculations.

Infrared: Fixed in commit 9b475ba7.

Cantina Managed: Verified the fix.

3.3.6 Unused Function and Commented Logic

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `InfraredMultisigGovernance` contract defines an `isContract` function, which checks whether a given address is a contract by inspecting its code length. In the `multiSendToken` function, there is a commented-out section that appears to have previously skipped token transfers to contract addresses. The rationale for this check is unclear, especially since the only token used in these scripts is `iBGT`, which is implemented using OpenZeppelin's standard `ERC20PresetMinterPauser` and does not include any hooks or custom logic on transfers.

> Historically, such checks are sometimes introduced to prevent unexpected behavior when interacting with contracts that implement hooks (e.g., ERC777 tokens or contracts with fallback functions that could re-enter or react to token transfers).

Recommendation: Remove the unused `isContract` function and any related commented-out code in `multiSendToken`. If there is no specific protocol-level reason to restrict transfers to contract addresses, such checks should not be present.

Infrared: Fixed in commit efd0ccff.

Cantina Managed: Verified the fix.